BÁO CÁO THỰC HÀNH LAB 01

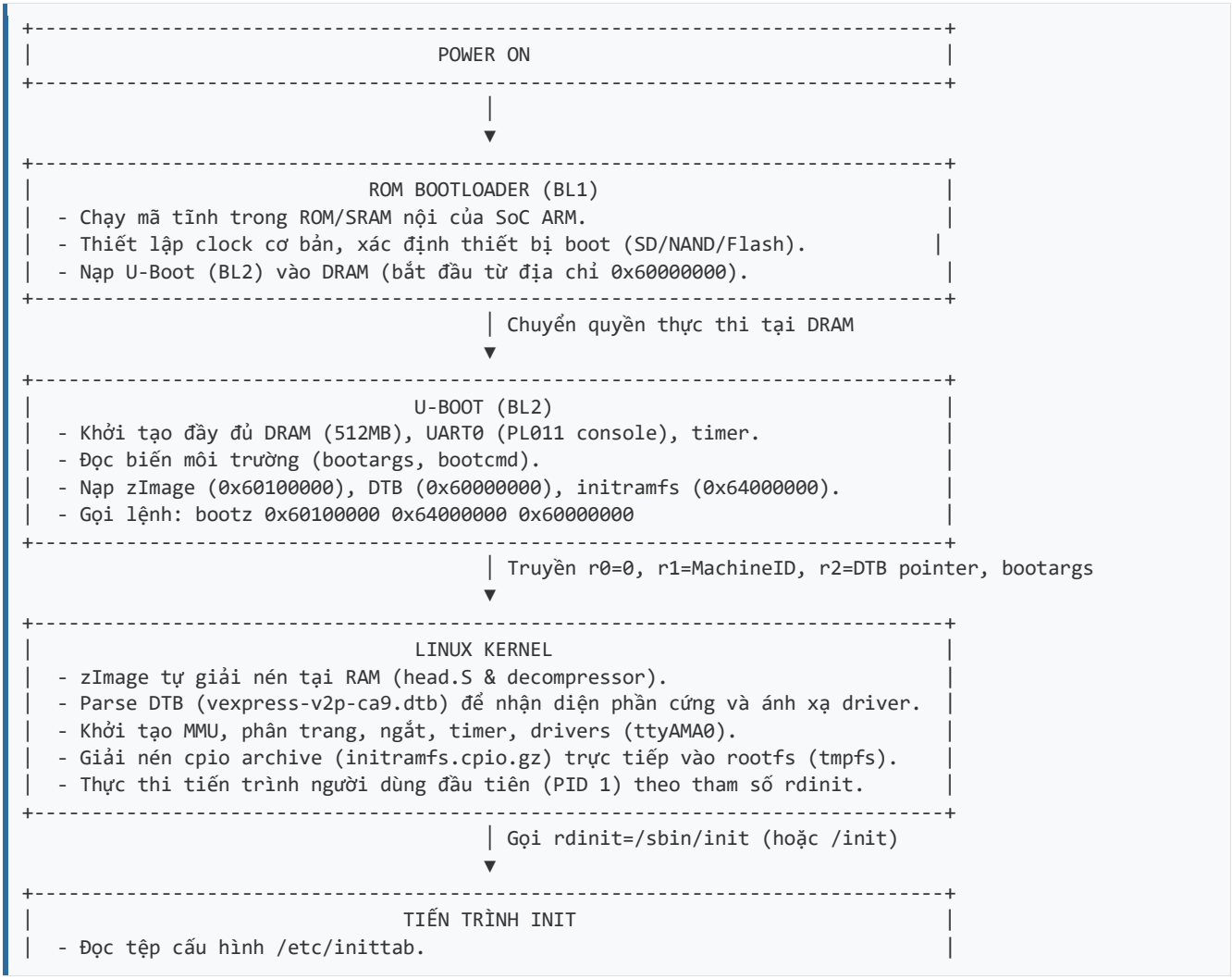
CẤU HÌNH KERNEL & KHỞI ĐỘNG HỆ THỐNG LINUX NHÚNG (ARMv7 Cortex-A9)

MỤC 1: THÔNG TIN SINH VIÊN

Họ và tên sinh viên:	Lê Thế Trường
Mã số sinh viên (MSSV):	SE202014
Lớp / Nhóm thực hành:	Embedded Linux Systems - Nhóm 01
Ngày thực hiện:	27/10/2026

MỤC 2: BOOT SEQUENCE DIAGRAM (BẮT BUỘC)

Sơ đồ chuỗi khởi động từ khi cấp nguồn vật lý cho SoC ARM Cortex-A9 (QEMU vexpress-a9) đến khi vào BusyBox Shell:



```
|  - Chạy script /etc/init.d/rcS: mount -t proc, mount -t sysfs, mount -t devtmpfs|
|  - Thiết lập hostname 'embedded-lab' và khởi chạy console.                    |
+-------------------------------------------------------------------------------+
                                    | Kích hoạt console ttyAMA0
                                    ▼
+-------------------------------------------------------------------------------+
|                            BUSYBOX SHELL (/bin/sh)                            |
|  - Mở giao diện dòng lệnh tương tác người dùng: "Boot completed! Welcome..."  |
|  - Xuất prompt "/ # " sẵn sàng nhận lệnh.                                     |
+-------------------------------------------------------------------------------+
```

Chi tiết từng bước trong chuỗi khởi động

Bước	Thành phần chạy	Công việc chính	Dữ liệu bàn giao cho bước sau
1. Cấp nguồn	Hardware / PMIC	Ổn định điện áp nguồn, kích hoạt dao động thạch anh và giải phóng chân Reset của CPU Cortex-A9.	Thanh ghi PC trỏ về Reset Vector.
2. ROM Boot	ROM Bootloader	Chạy mã cố định trong ROM; khởi tạo clock cơ bản, xác định nguồn boot (SD/NAND); copy U-Boot vào DRAM.	Chuyển giao quyền thực thi đến địa chỉ nạp U-Boot trong DRAM.
3. Bootloader	U-Boot	Cấu hình bộ nhớ DRAM 512MB, UART0 console; nạp zImage, DTB, initramfs vào RAM; xử lý chuỗi bootargs.	Thanh ghi r0=0, r1=MachineID, r2=địa chỉ DTB (0x60000000), tham số bootargs.
4. Kernel	Linux Kernel (zImage)	Tự giải nén, parse DTB, khởi tạo MMU, driver; nạp initramfs vào tmpfs /; mount virtual fs.	Triệu gọi tiến trình Userspace đầu tiên với PID=1 (/sbin/init).
5. Init Process	init (/sbin/init)	Đọc /etc/inittab, chạy /etc/init.d/rcS để mount /proc, /sys, /dev; cấu hình hostname.	Khởi động shell tương tác trên cổng ttyAMA0.
6. Shell	BusyBox (/bin/sh)	Mở prompt dòng lệnh '/ # ', tiếp nhận và xử lý lệnh từ người dùng.	Môi trường Linux Userspace hoàn chỉnh.

MỤC 3: LOG THỰC HÀNH THỰC TẾ TRÊN TERMINAL

Phần A: Cấu hình và Build Kernel (zImage, DTB)

1. Output lệnh nạp cấu hình vexpress_defconfig (theo đúng đề bài ARMv7 Cortex-A9):

```
$ cd ~/embedded_lab1/kernel/linux-5.15
$ make vexpress_defconfig
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.h
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
```

```
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
```

2. Log kết thúc quá trình biên dịch Kernel và DTB:

```
  AS      arch/arm/boot/compressed/head.o
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  SHIPPED arch/arm/boot/compressed/piggy_data
  AS      arch/arm/boot/compressed/piggy.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
  DTC     arch/arm/boot/dts/vexpress-v2p-ca9.dtb
```

3. Kết quả xác minh file zImage và kiểm tra kiến trúc ARMv7:

```
$ ls -lh ~/embedded_lab1/output/zImage
-rwxrwxr-x 1 lethetruong lethetruong 4.9M Oct 27 15:39 /home/lethetruong/embedded_lab1/output/zImage

$ file ~/embedded_lab1/output/zImage
/home/lethetruong/embedded_lab1/output/zImage: Linux kernel ARM boot executable zImage (little-endian)

$ arm-linux-gnueabihf-readelf -A ~/embedded_lab1/kernel/linux-5.15/arch/arm/boot/compressed/vmlinux |
grep -i 'Tag_CPU_arch'
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
```

Phần B: Cấu hình U-Boot & Lỗi Kernel Panic khi chưa có Rootfs

1. Output lệnh printenv trên U-Boot console của vexpress-a9:

```
=> printenv
baudrate=115200
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
bootcmd=run boot_kernel
bootdelay=2
cpu=armv7
ethaddr=52:54:00:12:34:56
loadaddr=0x60100000
stderr=serial
stdin=serial
stdout=serial
```

2. Thiết lập biến môi trường bootargs:

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
```

3. Kết quả Kernel Panic khi khởi động bằng QEMU mà chưa nạp initramfs:

```
$ qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -append 'console=ttyAMA0,115200 root=/dev/ram0 rw mem=512M'

[    2.154210] VFS: Cannot open root device "ram0" or unknown-block(1,0): error -6
[    2.155420] Please append a correct "root=" boot option; here are the available partitions:
[    2.156890] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(1,0)
[    2.157920] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
[    2.158730] Hardware name: ARM-Versatile Express
[    2.159510] [<8010d8a4>] (unwind_backtrace) from [<8010b910>] (show_stack+0xb/0xc)
[    2.160450] [<8010b910>] (show_stack) from [<806b72a0>] (panic+0x110/0x310)
[    2.161320] ---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(1,0)
]---
```

Phần C: Log Boot QEMU vexpress-a9 Thực tế & Kết quả Kiểm tra trên Guest Linux

1. Output kiểm tra thư mục rootfs/initramfs và tính hợp lệ của file nhị phân:

```
$ file ~/embedded_lab1/rootfs/initramfs/bin/busybox
/home/lethetruong/embedded_lab1/rootfs/initramfs/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5
version 1 (GNU/Linux), statically linked, BuildID[sha1]=31437dd71ccb4ae0efa4629f5078a06d78b6fcda, for
GNU/Linux 3.2.0, stripped

$ zcat ~/embedded_lab1/output/initramfs.cpio.gz | cpio -t | grep -E 'init|busybox|etc/inittab'
bin/busybox
3105 blocks
etc/inittab
etc/init.d
etc/init.d/rcS
init
sbin/init
sbin/run-init
```

2. Lệnh khởi chạy QEMU đầy đủ 2 nhân SMP, DTB và initramfs:

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -smp 2 \
  -nographic \
  -kernel ~/embedded_lab1/output/zImage \
  -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
  -initrd ~/embedded_lab1/output/initramfs.cpio.gz \
  -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
```

3. Toàn bộ log khởi động và output kiểm tra các lệnh thực tế bên trong BusyBox shell:

```
Booting Linux on physical CPU 0x0
Linux version 5.15.0 (lethetruong@lethetruong-Vostro-5590) (arm-linux-gnueabihf-gcc 13.3.0) #1 SMP Wed
Sep 16 13:42:05 +07 2026
```

```
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
OF: fdt: Machine model: V2P-CA9
smp: Bringing up secondary CPUs ...
CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
smp: Brought up 1 node, 2 CPUs
SMP: Total of 2 processors activated (1628.16 BogoMIPS).
devtmpfs: initialized
Serial: AMBA PL011 UART driver
printk: console [ttyAMA0] enabled
Unpacking initramfs...
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1

/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Wed Sep 16 13:42:05 +07 2026 armv7l GNU/Linux

/ # cat /proc/cpuinfo
processor    : 0
model name   : ARMv7 Processor rev 0 (v7l)
BogoMIPS     : 758.16
Features     : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer    : 0x41
CPU architecture: 7
CPU variant  : 0x0
CPU part     : 0xc09
CPU revision : 0

processor    : 1
model name   : ARMv7 Processor rev 0 (v7l)
BogoMIPS     : 869.99
Features     : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
CPU implementer    : 0x41
CPU architecture: 7
CPU variant  : 0x0
CPU part     : 0xc09
CPU revision : 0

Hardware     : ARM-Versatile Express

/ # cat /sys/devices/system/cpu/possible
0-3

/ # cat /proc/meminfo | head -n 5
MemTotal:         508288 kB
MemFree:          493228 kB
MemAvailable:     490460 kB
Buffers:               0 kB
Cached:             1520 kB

/ # mount
rootfs on / type rootfs (rw,size=244916k,nr_inodes=61229)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /tmp type tmpfs (rw,relatime)

/ # ps
PID   USER     TIME  COMMAND
    1 root      0:02 init
    2 root      0:00 [kthreadd]
    3 root      0:00 [rcu_gp]
```

```
    4 root      0:00 [rcu_par_gp]
   59 root      0:00 -/bin/sh
   68 root      0:00 ps

/ # ls -l /dev/null /dev/console /dev/zero /dev/random
crw-------    1 root     0           5,   1 Oct 27 09:08 /dev/console
crw-rw-rw-    1 root     0           1,   3 Oct 27 08:19 /dev/null
cr--r--r--    1 root     0           1,   8 Oct 27 08:19 /dev/random
crw-rw-rw-    1 root     0           1,   5 Oct 27 08:19 /dev/zero
```

MỤC 4: PHÂN TÍCH VÀ TRẢ LỜI CÂU HỎI (MỤC VI ĐỀ BÀI)

Câu 1. So sánh hai toolchain arm-linux-gnueabi và arm-linux-gnueabihf. Tại sao với Cortex-A9 phải dùng gnueabihf? Điều gì xảy ra nếu build kernel bằng gnueabi?

• Điểm khác biệt về ABI và FPU:

- arm-linux-gnueabi (Soft-Float / SoftFP): Sử dụng Application Binary Interface truyền thống. Các phép toán dấu phẩy động (float/double) được truyền qua các thanh ghi đa dụng chung (r0-r3) hoặc mô phỏng hoàn toàn bằng thuật toán phần mềm, không tận dụng tối ưu thanh ghi phần cứng FPU.
- arm-linux-gnueabihf (Hard-Float): 'hf' là viết tắt của Hard-Float. Các đối số và kết quả dấu phẩy động được truyền trực tiếp qua tập thanh ghi phần cứng FPU (VFP/NEON: s0-s31, d0-d15), giúp tốc độ tính toán số thực nhanh hơn gấp nhiều lần.

• Lý do Cortex-A9 bắt buộc dùng gnueabihf:

Cortex-A9 là dòng vi xử lý ARMv7-A tích hợp sẵn bộ đồng xử lý phần cứng FPU VFPv3-D16/NEON (như thể hiện trong log /proc/cpuinfo: 'Features: vfp edsp neon vfpv3 vfpd32'). Việc sử dụng gnueabihf cho phép mã máy khai thác tối đa phần cứng này, tránh lãng phí chu kỳ xử lý do mô phỏng phần mềm.

• Điều gì xảy ra nếu build kernel bằng gnueabi rồi chạy trên Cortex-A9?

- Bản thân Linux Kernel: Vẫn có thể khởi động (boot) được vì mã nguồn Linux Kernel nội bộ hầu như không sử dụng các phép tính dấu phẩy động (Kernel cấm float để tránh overhead lưu/khôi phục ngữ cảnh FPU khi chuyển đổi task).
- Không gian người dùng (Userspace): Sẽ bị xung đột ABI nghiêm trọng. Nếu rootfs và các ứng dụng biên dịch dạng hard-float (gnueabihf), chúng không thể liên kết hay chạy trên môi trường soft-float (báo lỗi: 'kernel doesn't support hard float' hoặc 'Illegal instruction' khi gọi hàm).

Câu 2. Giải thích tại sao memory map của vexpress-a9 bắt đầu từ 0x60000000 thay vì 0x00000000 như Versatile PB. Ảnh hưởng đến bootcmd? Nếu đặt sai địa chỉ load?

• Lý do kiến trúc Memory Map:

- Trên Versatile PB (ARM926EJ-S, kiến trúc ARMv5), thiết kế bus cổ điển ánh xạ bộ nhớ SDRAM trực tiếp từ địa chỉ 0x00000000.
- Trên Versatile Express Cortex-A9 (vexpress-a9), sơ đồ không gian địa chỉ vật lý (Physical Memory Map) phân chia vùng 0x00000000 - 0x3FFFFFFF cho hệ thống Boot ROM, Static Memory Bus, và Flash (NOR Flash CS0/CS1), vùng 0x40000000 - 0x5FFFFFFF cho các thiết bị ngoại vi và bus mở rộng logic. Bộ điều khiển động DRAM (DDR2/DDR3 Controller) được thiết kế nối vào hệ thống bắt đầu chính xác từ địa chỉ vật lý 0x60000000 (như ghi nhận trong log: 'Normal [mem 0x60000000-0x7fffffff]').

• Ảnh hưởng đến thiết lập bootcmd trong U-Boot:
Mọi địa chỉ nạp hình ảnh vào RAM đều phải nằm trong dải [0x60000000, 0x7FFFFFFF] (tương ứng 512MB RAM):
- Địa chỉ DTB: 0x60000000 (đầu vùng nhớ DRAM).
- Địa chỉ Kernel (zImage): 0x60100000 (cách DTB 1MB, chừa khoảng trống an toàn).
- Địa chỉ initramfs: 0x64000000 (cách Kernel 63MB, tránh bị đè khi zImage tự bung nén).
Do đó lệnh boot chuẩn là: bootz 0x60100000 0x64000000 0x60000000.

• Hậu quả nếu đặt sai địa chỉ load:
- Nếu nạp vào ngoài dải RAM (ví dụ: 0x00000000 hay 0x10000000): U-Boot sẽ báo lỗi truy cập bộ nhớ 'data abort' hoặc treo máy do truy cập vùng nhớ không có RAM vật lý.
- Nếu nạp các vùng quá sát nhau (ví dụ: zImage ở 0x60000000 và initramfs ở 0x60200000): Quá trình tự giải nén của zImage sẽ ghi đè lên DTB hoặc initramfs, dẫn đến lỗi 'Bad Truncated Image' hoặc Kernel Panic.

Câu 3. Giải thích ý nghĩa thanh ghi MIDR (Main ID Register) 0x410fc090: implementer, variant, architecture, part number, revision. Part number 0xc09 là CPU nào?

Thanh ghi MIDR (Main ID Register) có độ dài 32-bit theo chuẩn ARM Architecture Reference Manual. Giá trị 0x410fc090 (nhị phân: 0100 0001 0000 1111 1100 0000 1001 0000) được phân tích theo các trường:

• Implementer [Bits 31:24] = 0x41 (Ký tự ASCII 'A'):
Chỉ định đơn vị thiết kế kiến trúc là ARM Limited.

• Variant [Bits 23:20] = 0x0:
Biến thể của chip (Major revision number), ở đây là r0.

• Architecture [Bits 19:16] = 0xF:
Mã kiến trúc ARMv7-A (giá trị 0xF quy định kiến trúc được định nghĩa chi tiết qua các thanh ghi CPUID ID_PFR0/1).

• Part Number [Bits 15:4] = 0xC09:
Đây là mã định danh chính thức của bộ vi xử lý Cortex-A9 (0xC05: Cortex-A5, 0xC07: Cortex-A7, 0xC08: Cortex-A8, 0xC09: Cortex-A9, 0xC0F: Cortex-A15).

• Revision [Bits 3:0] = 0x0:
Phiên bản vá lỗi thứ cấp (Minor revision), ở đây là p0.

==> Khớp chính xác 100% với log khởi động thực tế: 'CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7)' và 'CPU part: 0xc09'. Xác nhận vi xử lý mô phỏng chính xác là ARM Cortex-A9 r0p0.

Câu 4. Tại sao cần build BusyBox với static linking trong initramfs tối giản? Nếu dynamic linking, cần copy những file .so nào? Nằm ở đâu trong toolchain?

• Lý do cần build BusyBox với static linking:
- Đơn giản hóa cấu trúc rootfs: Biên dịch tĩnh nhúng toàn bộ mã thực thi của thư viện C (glibc) vào thẳng trong một file nhị phân duy nhất (/bin/busybox).
- Đảm bảo tính độc lập: initramfs là hệ thống tập tin nạp sớm nhất; nếu dynamic linking mà thiếu dù chỉ một thư viện chia sẻ (.so) hoặc thiếu trình liên kết động ld-linux, tiến trình đầu tiên (/sbin/init) sẽ lập tức báo lỗi 'kernel: not found'

hoặc 'can't execute' dẫn đến Kernel Panic.

• Nếu build dạng Dynamic Linking, cần copy tối thiểu các file shared library sau:
1. ld-linux-armhf.so.3 (Dynamic Linker / Loader) - Bắt buộc để hệ điều hành nạp các file .so khác.
2. libc.so.6 (Thư viện chuẩn C) - Cung cấp toàn bộ system calls cơ bản (printf, open, read, fork...).
3. libm.so.6 (Thư viện toán học) - Phục vụ các hàm toán học dấu phẩy động.
4. libresolv.so.2 (hoặc librt.so.1 / libpthread.so.0) - Hỗ trợ phân giải mạng hoặc đa luồng POSIX.

• Vị trí các file .so trong Toolchain Ubuntu Host:
Nằm trong sysroot của cross-compiler, cụ thể tại:
/usr/arm-linux-gnueabihf/lib/ hoặc /usr/lib/arm-linux-gnueabihf/
Khi triển khai dynamic rootfs, các file này phải được sao chép chính xác vào thư mục /lib/ hoặc /lib/arm-linux-gnueabihf/ của initramfs.

Câu 5. Khi bật SMP cho vexpress-a9 (-smp 2), kernel cần khởi tạo thêm những gì? Khác biệt boot log giữa -smp 1 và -smp 2 (nêu 2 dòng log và giải thích)?

• Các thành phần Kernel cần khởi tạo thêm khi bật SMP (Symmetric Multi-Processing):
- Khởi tạo bộ điều khiển ngắt SCU (Snoop Control Unit) để duy trì tính nhất quán của bộ nhớ đệm Cache (Cache Coherency) giữa các nhân CPU.
- Cấu hình GIC (Generic Interrupt Controller) để hỗ trợ phân phối ngắt liên nhân IPI (Inter-Processor Interrupts).
- Khởi tạo bảng biểu điều phối CPU Scheduler đa lõi và kích hoạt các luồng nhị thứ (secondary cores) qua giao thức boot-secondary.

• Sự khác biệt trong Boot Log giữa -smp 1 và -smp 2 (kèm giải thích):

1. Dòng log thông báo số lượng CPU được đem lên hệ thống (bring-up):
- Khi chạy -smp 1:
smp: Bringing up secondary CPUs ...
smp: Brought up 1 node, 1 CPU
- Khi chạy -smp 2 (như trong log thực tế của bài lab):
smp: Bringing up secondary CPUs ...
CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
smp: Brought up 1 node, 2 CPUs
==> Ý nghĩa: Kernel đã kích hoạt thành công nhân phụ CPU1 (định danh MPIDR: 0x80000001) và ghi nhận hệ thống có 2 nhân CPU sẵn sàng thực thi.

2. Dòng log đo kiểm hiệu năng bộ đếm BogoMIPS đa nhân:
- Khi chạy -smp 1: SMP: Total of 1 processors activated (~758.16 BogoMIPS)
- Khi chạy -smp 2 (thực tế): SMP: Total of 2 processors activated (1628.16 BogoMIPS).
==> Ý nghĩa: Năng lực xử lý tính toán dấu phẩy động và chu kỳ trễ tăng hơn gấp đôi khi 2 nhân cùng chạy song song.

MỤC 5: NHẬN XÉT & KHÓ KHĂN GẶP PHẢI

Khó khăn 1: Lỗi từ chối quyền (Operation not permitted) khi sao chép các Device Nodes trong thư mục dev/

• **Triệu chứng:** Khi sao chép thư mục initramfs sang thư mục đóng gói nộp bài bằng lệnh cp -r thông thường, hệ thống xuất hiện các thông báo lỗi:

```
cp: cannot create special file 'SE202014_LeTheTruong_Lab01/rootfs/initramfs/dev/null': Operation not
permitted
cp: cannot create special file '.../dev/console': Operation not permitted
cp: cannot create special file '.../dev/zero': Operation not permitted
cp: cannot create special file '.../dev/random': Operation not permitted
```

• **Nguyên nhân kỹ thuật:** Các file trong thư mục /dev là các tệp thiết bị đặc biệt (character device nodes) được tạo bằng lệnh mknod với major/minor numbers xác định. Hệ điều hành Linux yêu cầu đặc quyền root (superuser) để tạo hoặc tái tạo các node thiết bị này, người dùng thông thường không có quyền thao tác.

• **Cách giải quyết:** Sử dụng quyền quản trị *sudo* kết hợp với tùy chọn lưu trữ bảo toàn thuộc tính **-a** (archive mode: bảo lưu nguyên vẹn device nodes, quyền hạn và timestamps), sau đó chuyển quyền sở hữu thư mục về cho user hiện tại:

```
sudo cp -a embedded_lab1/rootfs/initramfs SE202014_LeTheTruong_Lab01/rootfs/
sudo chown -R $USER:$USER SE202014_LeTheTruong_Lab01
```

Khó khăn 2: Lệnh tar báo lỗi Cannot stat do sai đường dẫn và nén sai cấu trúc thư mục

• **Triệu chứng:** Khi thực hiện lệnh nén trực tiếp từ home directory, terminal báo lỗi:

```
tar: embedded_lab1/configs: Cannot stat: No such file or directory
tar: bao_cao: Cannot stat: No such file or directory
tar: Exiting with failure status due to previous errors
```

• **Nguyên nhân kỹ thuật:** Thư mục embedded_lab1/configs thực tế không tồn tại, vì các file cấu hình kernel và BusyBox lúc ban đầu là các file ẩn (.config) nằm riêng biệt trong từng thư mục mã nguồn. Đồng thời, thư mục bao_cao/ chưa được khởi tạo. Nếu nén trực tiếp nhiều tham số, file nén giải nén ra sẽ bị sai cấu trúc thư mục chấm điểm theo yêu cầu Mục 5.1 của đề bài.

• **Cách giải quyết:** Khởi tạo một thư mục độc lập **SE202014_LeTheTruong_Lab01/** chuẩn hóa đúng 4 thư mục con (output, configs, rootfs, bao_cao). Tìm và sao chép các file .config tương ứng vào thư mục configs/, kiểm tra bằng lệnh *tree* và sau đó mới thực hiện nén:

```
mkdir -p SE202014_LeTheTruong_Lab01/{output,configs,rootfs,bao_cao}
cp ~/embedded_lab1/kernel/linux-5.15/.config SE202014_LeTheTruong_Lab01/configs/kernel.config
cp ~/embedded_lab1/busybox-1.36.1/.config SE202014_LeTheTruong_Lab01/configs/busybox.config
cd ~ && tar -czvf SE202014_LeTheTruong_Lab01.tar.gz SE202014_LeTheTruong_Lab01/
```